

ЛАБОРАТОРНАЯ РАБОТА №6 ОТЛАДКА ПРИЛОЖЕНИЯ LOGCAT, ТОЧКИ ОСТАНОВА

Как разметка связана с кодом?

Программисты пишут код с ошибками всегда – и чем быстрее вы найдете их, тем вы круче как программист.

Сегодня осваиваем два инструмента: **Logcat** показывает, что происходило в приложении, а **отладчик** позволяет остановить приложение на конкретной строке и заглянуть в значения переменных в этот момент.

Часть 1. Приложение “Угадай число”

Классическая игра: компьютер загадывает число от 1 до 100, игрок вводит догадки, приложение подсказывает – больше или меньше и считает попытки.

Полный файл **activity_main.xml** скопируйте его (полностью набирать XML руками не будем, в этом нет смысла):

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Угадай число"
        android:textSize="24sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:textSize="18sp" />

    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:hint="Твоя догадка"
        android:inputType="number" />

    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Проверить" />

    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Новая игра" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:textSize="16sp" />
</LinearLayout>
```

Обратите внимание у **EditText** есть –

```
android:inputType="number"
```

это значит, что при вводе клавиатура будет **цифровой** – это защита от букв, но она не спасет от всех проблем (увидим в **части 3**).

А теперь:

1. вынесите все строки из элементов в ресурсы **strings.xml**.
2. назначьте каждому элементу свой **id**, например:

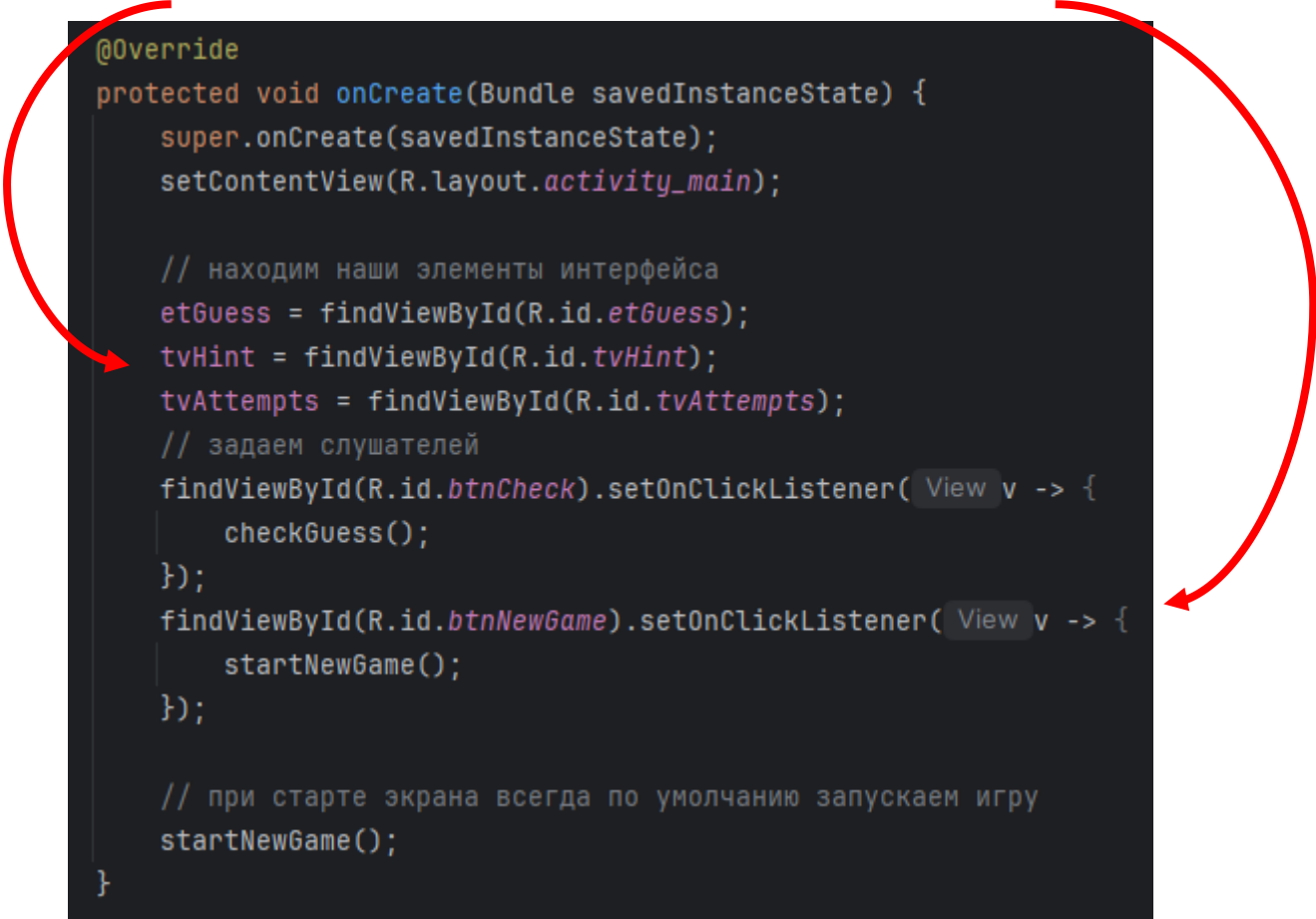
```
android:id="@+id/btnOrder"
```

Часть 2. Оживляем кнопку: слушатели

Сначала создаем поля класса для элементов, с которыми будем работать, и реализуем логику нашей игры:

```
12 </> public class MainActivity extends AppCompatActivity {
13
14     private int secret;    // загаданное число 4 usages
15     private int attempts; // количество попыток 6 usages
16
17     private EditText etGuess; 2 usages
18     private TextView tvHint; 5 usages
19     private TextView tvAttempts; 3 usages
20
21     @Override
22     > protected void onCreate(Bundle savedInstanceState) {...}
23
24
41
42     private void startNewGame() { 2 usages
43         // загадываем число
44         Random random = new Random();
45         secret = random.nextInt( bound: 100) + 1; // случайное число от 1 до 100
46         attempts = 0;
47         // обновляем текст
48         tvHint.setText("Я загадал число от 1 до 100. Попробуй угадать!");
49         tvAttempts.setText("Попыток: " + attempts);
50
51         Log.d( tag: "Game", msg: "Новая игра, загадано: " + secret);
52     }
53
54     private void checkGuess() { 1 usage
55         // получаем введенное число
56         String input = etGuess.getText().toString();
57         int guess = Integer.parseInt(input);
58
59         attempts++;
60         tvAttempts.setText("Попыток: " + attempts);
61
62         if (guess == secret) {
63             tvHint.setText("Верно! Число угадано. Попыток: " + attempts);
64             Log.i( tag: "Game", msg: "Число угадано за " + attempts + " попыток");
65         } else if (guess < secret) {
66             tvHint.setText("Не угадал. Загаданное число БОЛЬШЕ");
67         } else {
68             tvHint.setText("Не угадал. Загаданное число МЕНЬШЕ");
69         }
70     }
71 }
```

Затем **найдем наши элементы** на экране по их id, и **зададим слушателей**:



```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    // находим наши элементы интерфейса
    etGuess = findViewById(R.id.etGuess);
    tvHint = findViewById(R.id.tvHint);
    tvAttempts = findViewById(R.id.tvAttempts);
    // задаем слушателей
    findViewById(R.id.btnCheck).setOnClickListener( View v -> {
        checkGuess();
    });
    findViewById(R.id.btnNewGame).setOnClickListener( View v -> {
        startNewGame();
    });

    // при старте экрана всегда по умолчанию запускаем игру
    startNewGame();
}
```

В коде было два новых вызова:

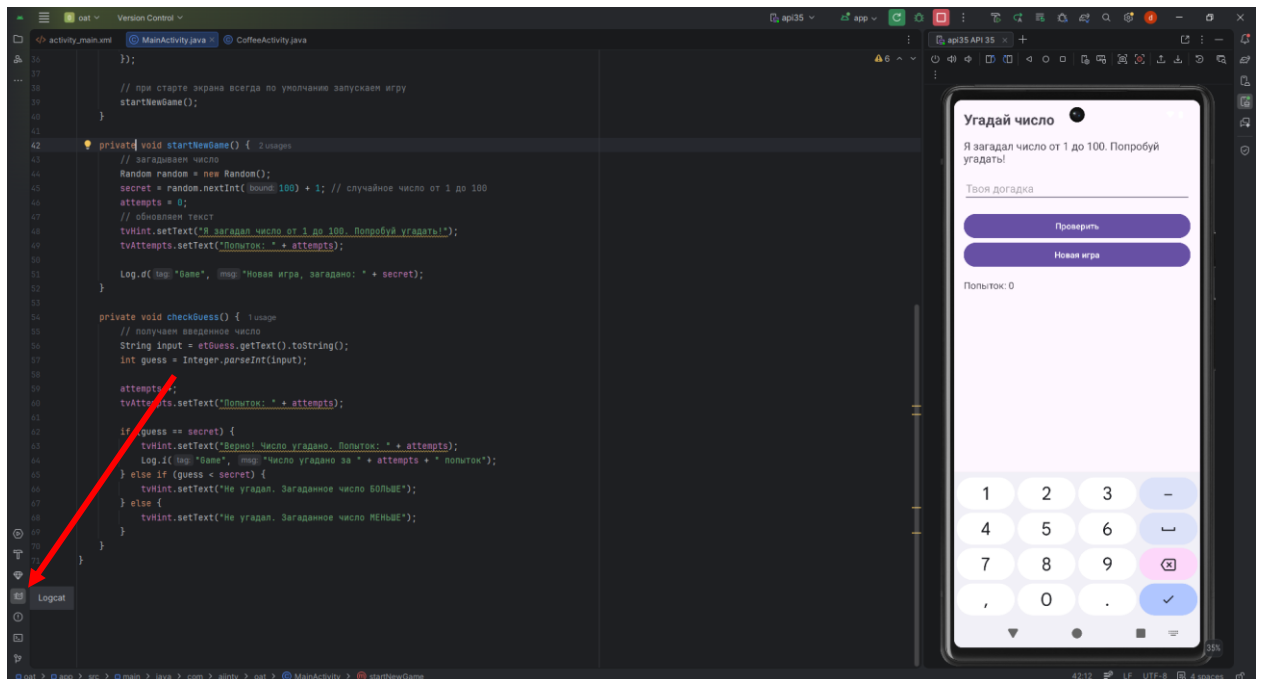
1. **random.nextInt(100)** – даёт число от 0 до 99 (не включая 100), поэтому + 1 сдвигает до 1–100
2. **Integer.parseInt(input)** – превращает строку "50" в число 50 – и это самое опасное место программы, хотя пока выглядит безобидно.

Сыграйте пару игр и выиграйте. (Подсказка: загаданное число пишется в **Logcat**)

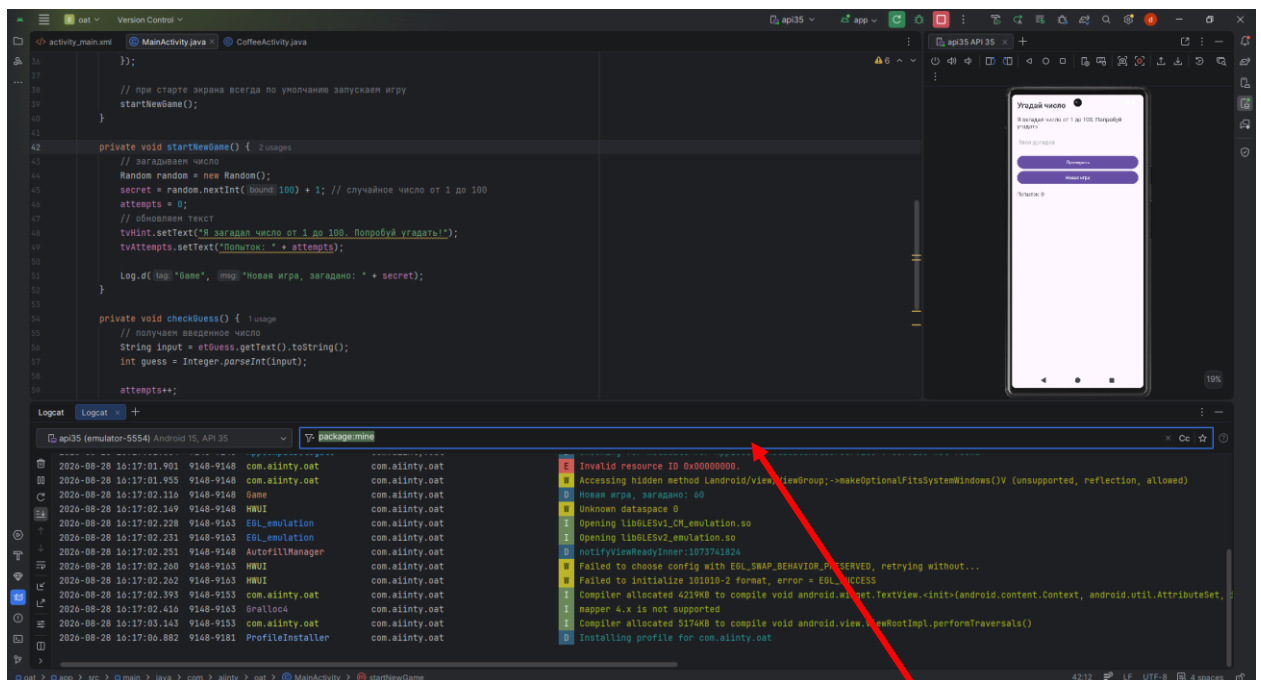
ваыыы

Часть 3. Logcat: журнал событий приложения

Откройте вкладку **Logcat** внизу.



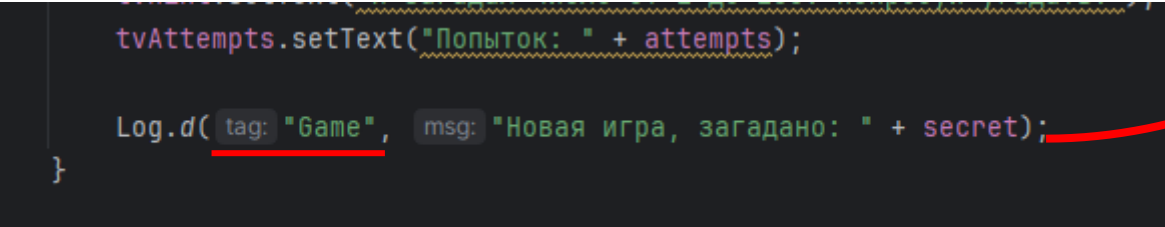
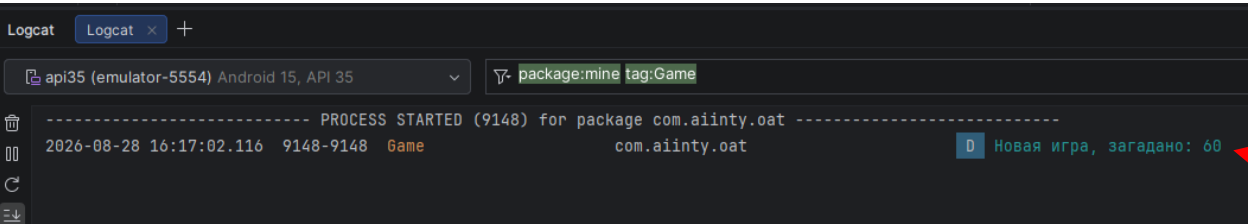
Вы видите поток сообщений **всей системы**: там пишут и ваше приложение, и сама Android, и другие службы – очень много строк в минуту.



Главные **фильтры**, которые можно комбинировать (поле поиска сверху):

1. **package:mine** – только ваше приложение
2. **tag:Game** – только сообщения с тегом Game

Найдите свои сообщения, отфильтровав по **tag:Game**:



Уровень	Метод	Когда используют
D (Debug)	<u>Log.d()</u>	отладочные сообщения
I (Info)	<u>Log.i()</u>	заметные события (но не ошибочные)
W (Warning)	<u>Log.w()</u>	странно, но не смертельно
E (Error)	<u>Log.e()</u>	ошибка

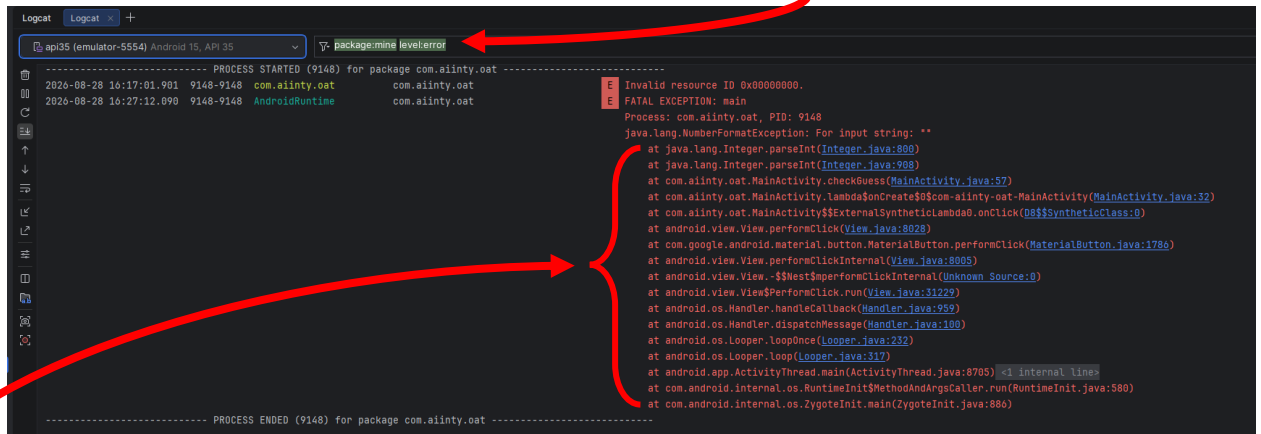
В **checkGuess()** после разбора числа добавьте предупреждение, если игрок вышел из диапазона (либо меньше нижней границы, либо больше верхней)

Проверьте: введите 150 и найдите в Logcat предупреждение

Часть 4. Читаем большие страшные ошибки

Уроните приложение: нажмите “Проверить” с пустым полем ввода.

Приложение закрывается. В **Logcat** отфильтруйте level:error. Вы увидите примерно это:



Давайте разберем эту ошибку сверху вниз:

1. **FATAL EXCEPTION: main** – приложение аварийно завершилось в главном потоке
2. **java.lang.NumberFormatException: For input string: ""** это – “тип.Ошибки: Её сообщение”. Тип говорит, что сломалось (попытка превратить строку в число), сообщение это конкретика: системе скормили пустую строку "". Сообщение читайте всегда – часто в нём сразу ответ.
3. И огромная тьма строк дальше – **Стек вызовов** – список строк вида в каком файле, в каком методе, на какой строке. Читается снизу вверх: что за чем вызывалось.

НО большинство строк в стеке – **не ваши** (java.lang.Integer.parseInt это внутренности Java). Потому ищите в стеке строки со своим пакетом (у меня это com.aiinty.oat, у вас может быть com.example.myapplication или что-либо другое)

Почините – вылет происходит потому, что **parseInt** не может перевести пустую строку в число. Надо добавить проверку на пустоту (**input.isEmpty()**)

Запустите, и снова нажмите с пустым полем: приложение живое.

Потому надо запомнить первое правило леса – никогда нельзя доверять данным от пользователя без проверки.

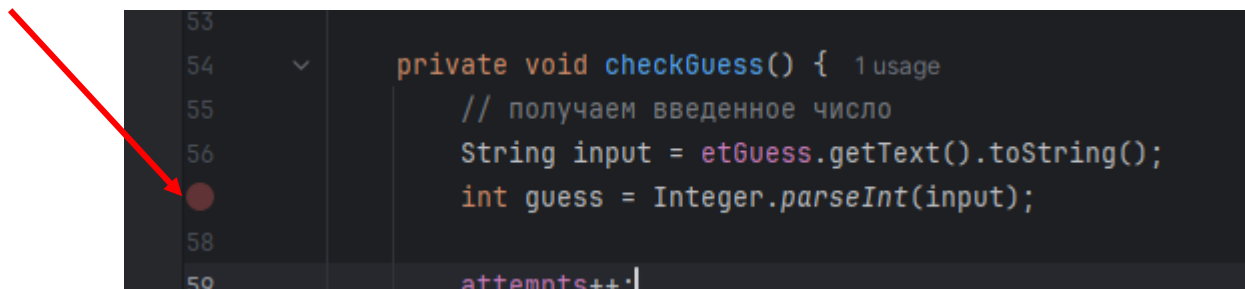
Часть 5. Точки останова: останавливаем время

Logcat рассказывает о прошлом. Однако **отладчик** позволяет заглянуть в настоящее: остановить приложение на выбранной строке и рассмотреть значения всех переменных.

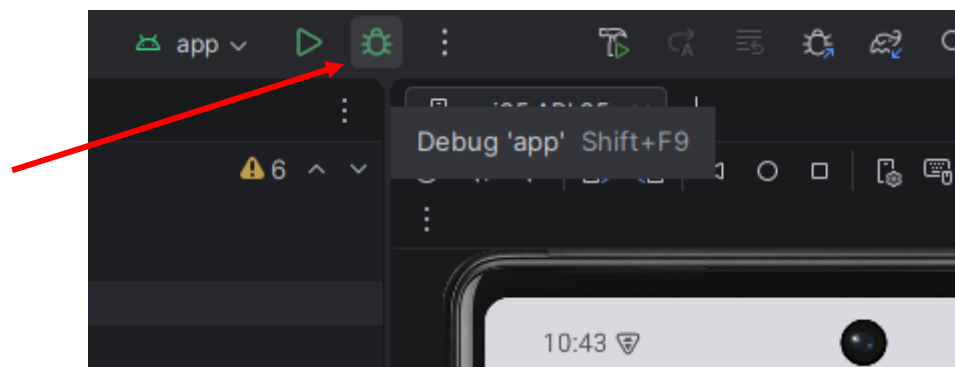
Именно так ищут всякие логические ошибки – когда приложение **не падает**, но работает **неправильно**.

Порядок работы:

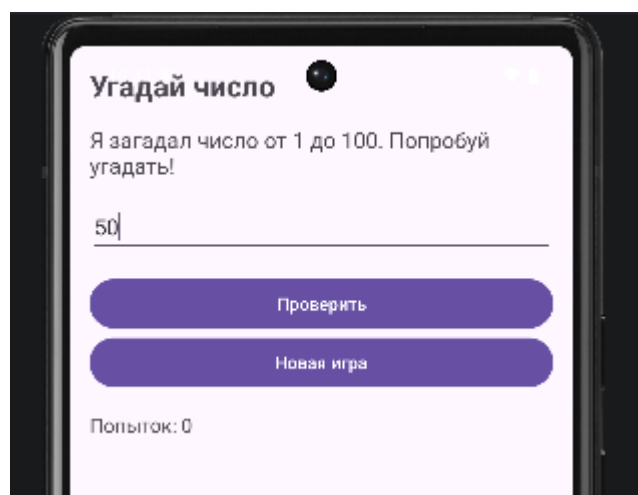
1. **Поставьте точку останова:** нажмите слева от строки `int guess = Integer.parseInt(input);` - появится красный кружок и строка станет красной.



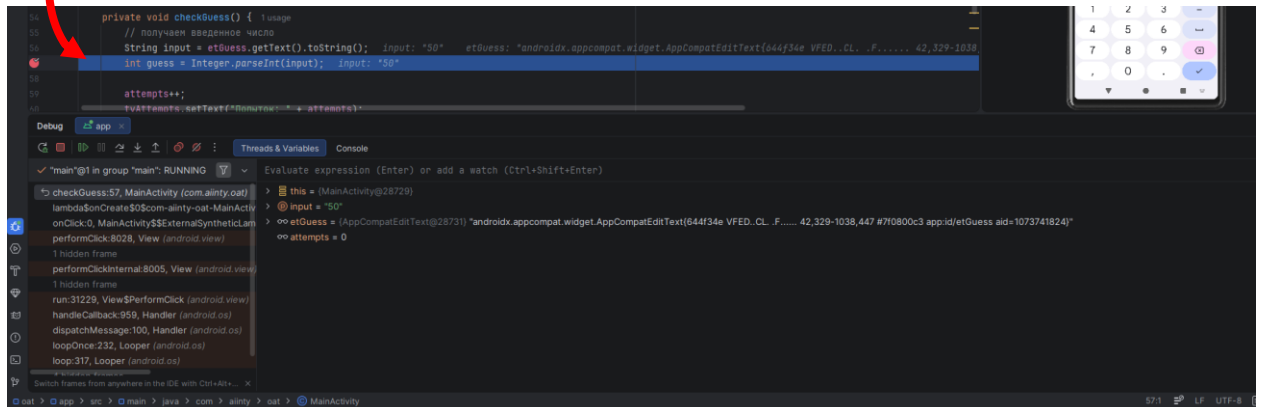
2. **Запустите приложение кнопкой с жучком (Debug).** – Обычный Run тоже подойдёт для игры, но точки останова при нём не срабатывают



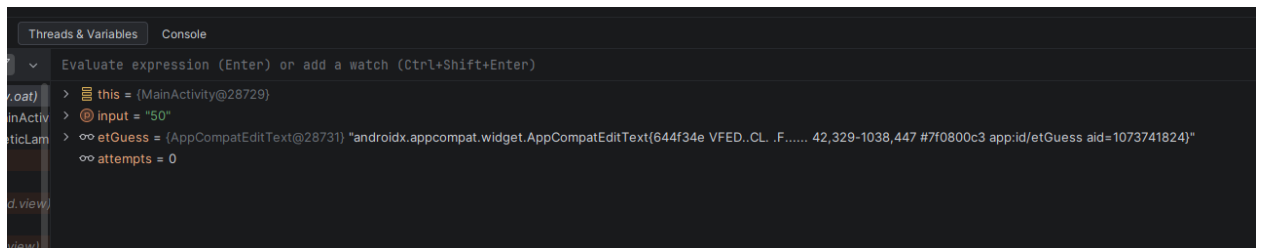
3. **Введите число** (например, 50) и нажмите – Проверить



4. **Android Studio перехватит выполнение:** нужная строка подсвечена голубым это значит, что приложение замерло именно здесь, **строка ещё не выполнена**

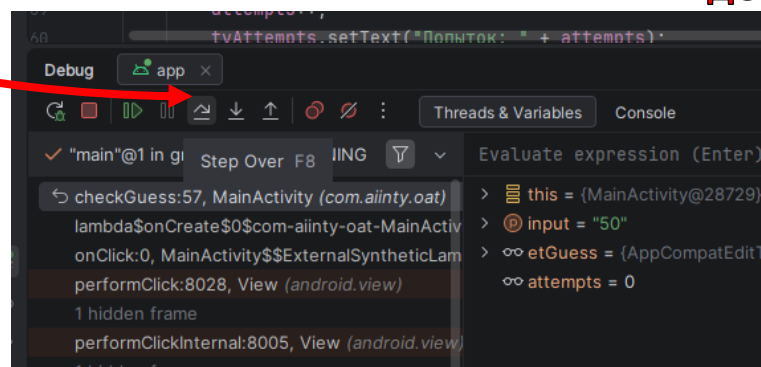


5. Посмотрите на панель **Variables:** там актуальные значения всех переменных прямо сейчас – input, а после шага и остальные. Те же значения подсвечены серым прямо в коде.

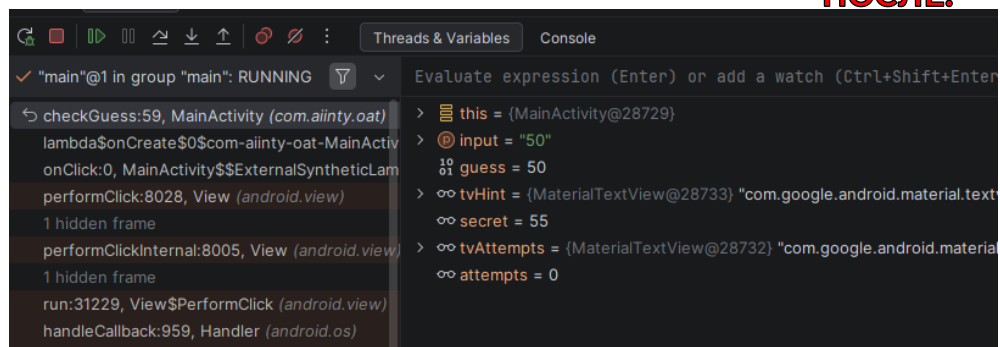


6. Жмите **F8 (Step Over)** и следите: **guess** получило значение. Так вы буквально видите ход мысли программы.

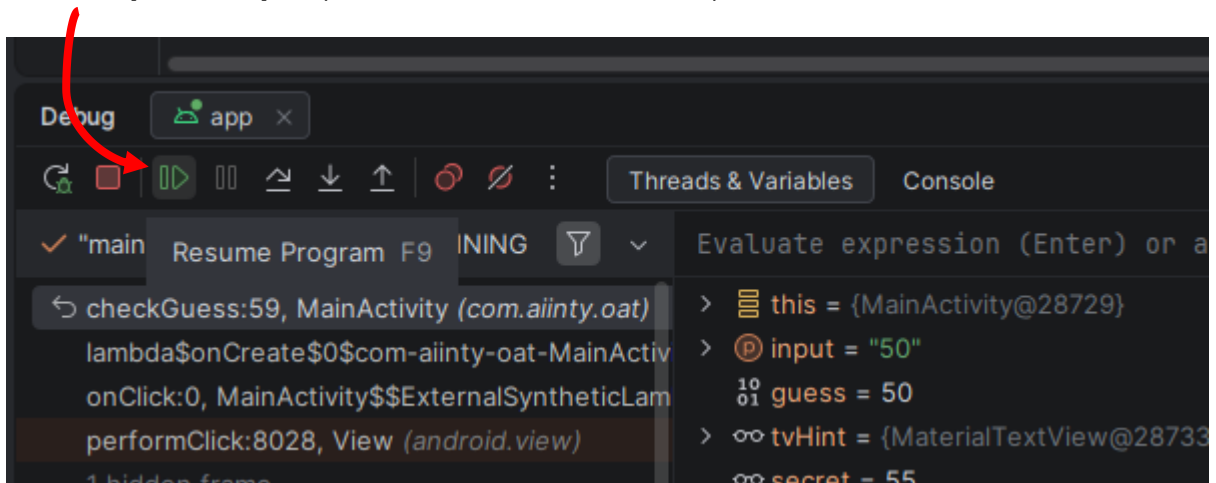
ДО:



ПОСЛЕ:



7. Попробуйте еще несколько раз понажимать **F8 (Step Over)** и посмотрите, что будет с приложением.
8. **F9 (Resume)** – приложение оживает и дорабатывает само



ЫВАЫВА

Требования к итоговой версии

1. Игра **работает** полностью: подсказки, счётчик попыток и новая игра
2. Пустой ввод **не роняет** приложение
3. В коде есть **Log.d** при старте игры, **Log.i** при победе, **Log.w** при вводе вне диапазона

Дополнительно (по желанию)

1. **Условная точка останова** – правый клик по красному кружку -> поле Condition -> введите attempts>3. Запустите отладку: сработает только на четвёртой попытке.